

2η ΕΡΓΑΣΙΑ ΣΤΗΝ ΨΗΦΙΑΚΗ ΕΠΕΞΕΡΓΑΣΙΑ ΕΙΚΟΝΑΣ

ΟΝΟΜΑΤΕΠΩΝΥΜΟ: ΡΟΥΣΟΣ ΣΤΑΜΑΤΙΟΣ

AEM: 10704

1. ΕΙΣΑΓΩΓΗ

Στην παρούσα εργασία εξετάζουμε τεχνικές εντοπισμού ακμών και κυκλικών σχημάτων σε εικόνες, χρησιμοποιώντας δύο γνωστές μεθόδους: τον αλγόριθμο Sobel, και τον Laplacian of Gaussian (LoG), σε συνδυασμό με τη μέθοδο Hough για κύκλους.

Οι μέθοδοι που χρησιμοποιούνται σε αυτή την εργασία εντοπίζουν περιοχές με έντονες μεταβολές φωτεινότητας, οι οποίες αντιστοιχούν σε φυσικά όρια αντικειμένων.

Η μέθοδος Hough εφαρμόζεται έπειτα για να εντοπίσει κυκλικά σχήματα στην εικόνα, αξιοποιώντας τις ακμές που προέκυψαν από τις παραπάνω μεθόδους. Εξετάζουμε πώς διάφορες παράμετροι επηρεάζουν την ποιότητα των αποτελεσμάτων, και αναλύουμε συγκριτικά τις επιδόσεις των δύο τεχνικών εντοπισμού ακμών.

Για την εργασία χρησιμοποιήθηκε η 3.13.3 έκδοση της Python στο VScode.

2. Μέθοδος SOBEL

Ο αλγόριθμος Sobel αποτελεί μια από τις πιο απλές και ευρέως χρησιμοποιούμενες τεχνικές για την ανίχνευση ακμών σε εικόνες. Βασίζεται στον υπολογισμό της πρώτου παραγώγου της φωτεινότητας, εφαρμόζοντας δύο μάσκες Sobel: μία για την οριζόντια και μία για την κατακόρυφη κατεύθυνση.

Οι μάσκες Sobel έχουν την παρακάτω μορφή:

$$G_x = \begin{bmatrix} -1 & 0 & +1 \end{bmatrix}$$

$$\begin{bmatrix} -2 & 0 & +2 \end{bmatrix}$$

$$\begin{bmatrix} -1 & 0 & +1 \end{bmatrix}$$

$$G_y = \begin{bmatrix} +1 & +2 & +1 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} -1 & -2 & -1 \end{bmatrix}$$

Η ένταση της ακμής σε κάθε σημείο (n_1, n_2) υπολογίζεται με βάση το μέτρο του διανύσματος κλίσης:

$$|G(n_1, n_2)| = \sqrt{g_1(n_1, n_2)^2 + g_2(n_1, n_2)^2}$$

Όπου g_1 και g_2 είναι οι εικόνες που προκύπτουν από τη συνέλιξη της αρχικής εικόνας με τις μάσκες Sobel κατά x και y αντίστοιχα.

Οι περιοχές της εικόνας όπου η τιμή του $g(n_1, n_2)$ είναι μεγάλη, αντιστοιχούν σε σημεία με έντονες μεταβολές φωτεινότητας - δηλαδή ακμές.

Επιλογή Threshold

Δοκιμάστηκαν διάφορες τιμές threshold και διαπιστώθηκε ότι τιμές κοντά στο 0.5 απέδωσαν καλύτερα σε εικόνες υψηλής ανάλυσης, καθώς διατηρούσαν τις βασικές ακμές (ρίγες μπάλας, περίγραμμα), μειώνοντας τον θόρυβο.

ΓΡΑΦΙΚΗ ΠΑΡΑΣΤΑΣΗ THRESHOLDS - ΑΝΙΧΝΕΥΟΜΕΝΑ ΣΗΜΕΙΑ

Για την αξιολόγηση της επίδρασης της τιμής κατωφλίου (**threshold**) στο αποτέλεσμα της μεθόδου Sobel, δημιουργήθηκε γραφική παράσταση που απεικονίζει τον αριθμό των ανιχνευμένων ακμών συναρτήσει του threshold. Παρατηρείται ότι:

- Για μικρές τιμές **threshold** (π.χ. 0.1–0.3), εντοπίζεται **μεγάλος αριθμός ακμών**, περιλαμβάνοντας τόσο τις σημαντικές όσο και πολλές θορυβώδεις ακμές.
- Καθώς το **threshold** αυξάνεται, ο αριθμός των ανιχνευόμενων ακμών **μειώνεται σταδιακά**, καθώς αγνοούνται πιο αδύναμα σημεία κλίσης.

- Για $\text{threshold} > 0.6$, παραμένουν μόνο οι **έντονες ακμές**, κυρίως στα περιγράμματα της μπάλας και των μαύρων γραμμών.

Η γραφική παράσταση βοηθά στη **ρύθμιση του κατάλληλου threshold**, ώστε να επιτυγχάνεται ισορροπία μεταξύ πληρότητας και αποφυγής θορύβου. Στην εργασία, επιλέχθηκε τιμή **threshold = 0.5**, που αποδίδει ικανοποιητικά αποτελέσματα.

3. ΜΕΘΟΔΟΣ LoG

Η μέθοδος **Laplacian of Gaussian (LoG)** χρησιμοποιείται για την ανίχνευση ακμών μέσω του εντοπισμού περιοχών της εικόνας όπου η **δεύτερη παράγωγος** της φωτεινότητας παρουσιάζει απότομες μεταβολές. Αυτό επιτυγχάνεται εφαρμόζοντας ένα φίλτρο Laplacian μετά από εξομάλυνση της εικόνας με φίλτρο Gaussian. Οι ακμές εντοπίζονται με τη μέθοδο **zero-crossing**, δηλαδή όπου το σήμα αλλάζει πρόσημο από θετικό σε αρνητικό ή το αντίστροφο.

Ωστόσο, η μέθοδος είναι **ευαίσθητη στο θόρυβο**, ειδικά στο εσωτερικό αντικειμένων ή σε υφές, όπου μικρές διακυμάνσεις στη φωτεινότητα μπορούν να προκαλέσουν ψευδείς εντοπισμούς ακμών. Για να αντιμετωπίσουμε αυτό το φαινόμενο:

- Εφαρμόσαμε **Gaussian blur** (με `cv2.GaussianBlur`) πριν τη χρήση του LoG στο demo. Το Gaussian blur μειώνει τον υψηλής συχνότητας θόρυβο και εξομαλύνει την εικόνα, διατηρώντας παράλληλα τις σημαντικές μεταβολές έντασης στις ακμές. Παρόλο που η LoG μάσκα περιλαμβάνει εξομάλυνση τύπου Gaussian, εφαρμόστηκε επιπλέον Gaussian blur στην είσοδο για να ενισχυθεί ο καθαρισμός περιοχών υψηλού θορύβου, καθώς με την 5x5 μάσκα που δόθηκε υπήρχε πολύς θόρυβος στο εσωτερικό της μπάλας.
- Επιπλέον, κατά τη διαδικασία εντοπισμού των zero-crossings, προσθέσαμε έναν **πρόσθετο έλεγχο** ώστε να θεωρείται έγκυρη μια ακμή μόνο όταν η διαφορά μέγιστης και ελάχιστης τιμής στο τοπικό παράθυρο είναι αρκετά μεγάλη. Συγκεκριμένα, εφαρμόσαμε την εξής συνθήκη:

```
if patch.max() > 0 and patch.min() < 0 and (patch.max() - patch.min()) > 0.5:  
    out_img_array[i, j] = 1
```

Με αυτόν τον τρόπο, απορρίψαμε αδύναμες μεταβολές φωτεινότητας που οφείλονταν σε θόρυβο, διατηρώντας μόνο τις σημαντικές ακμές.

4. HOUGH

Η μέθοδος **Hough Transform** αποτελεί έναν από τους πιο κλασικούς και ισχυρούς αλγορίθμους για την ανίχνευση γεωμετρικών σχημάτων σε εικόνες, όπως ευθείες και κύκλοι. Στην παρούσα εργασία, εφαρμόσαμε την **Circular Hough Transform** για τον εντοπισμό κύκλων.

Η βασική ιδέα πίσω από την ανίχνευση κύκλων με Hough είναι ότι κάθε ακμιακό σημείο (edge point) μπορεί να είναι μέρος ενός ή περισσότερων κύκλων. Ο κύκλος περιγράφεται με την εξίσωση:

$$(x - a)^2 + (y - b)^2 = r^2$$

όπου (a,b) είναι οι συντεταγμένες του κέντρου και r η ακτίνα.

Διαδικασία

- Αρχικά, εξάγουμε τις ακμές της εικόνας είτε με τη μέθοδο **Sobel** είτε με τη **LoG**, ώστε να περιορίσουμε την αναζήτηση του Hough μόνο στα σημαντικά σημεία.
- Κάθε pixel που χαρακτηρίζεται ως ακμή συμβάλλει με **ψήφους** σε έναν τρισδιάστατο συσσωρευτή (accumulator), ο οποίος καταγράφει πιθανές τοποθεσίες κύκλων — δηλαδή κέντρα (a,b) και ακτίνες r.
- Συγκεκριμένα, για κάθε ακμιακό pixel, ο αλγόριθμος υπολογίζει όλους τους δυνατούς κύκλους στους οποίους θα μπορούσε να ανήκει, και για κάθε τέτοιο υποψήφιο κύκλο προστίθεται μία ψήφος στο αντίστοιχο κελί του συσσωρευτή. Όσο περισσότερα pixels "συμφωνούν" σε έναν συγκεκριμένο κύκλο, τόσο περισσότερες ψήφους συγκεντρώνει.
- Για να θεωρηθεί ότι ένας κύκλος **όντως εντοπίστηκε**, πρέπει να έχει **αρκετή υποστήριξη** από τα ακμιακά pixels. Αυτό ελέγχεται με βάση ένα **κατώφλι ψήφων V_min**:

Ένας κύκλος θεωρείται έγκυρος μόνο αν το πλήθος των ψήφων του είναι μεγαλύτερο ή ίσο με το V_min.

- Έτσι, το $V_{\min}$ λειτουργεί σαν **μηχανισμός φιλτραρίσματος** των αδύναμων ή λανθασμένων ανιχνεύσεων. Χαμηλές τιμές επιτρέπουν την ανίχνευση πολλών, αλλά ενδεχομένως θορυβωδών κύκλων, ενώ υψηλότερες τιμές εστιάζουν μόνο στους πιο έντονους και σταθερούς κύκλους.

Παράμετροι

Κατά την υλοποίηση της συνάρτησης `circ_hough`, χρησιμοποιήσαμε τις παρακάτω βασικές παραμέτρους:

- $R_{\max}$: η μέγιστη ακτίνα κύκλου που εξετάζουμε.
- `dim`: πίνακας με τρεις τιμές που καθορίζουν το μέγεθος του συσσωρευτή. Συγκεκριμένα, το πλήθος διακριτών τιμών για x , y και r .
- $V_{\min}$: κατώφλι ψήφων για την αποδοχή ενός κύκλου.

Μεγαλύτερη τιμή $V_{\min}$ αυξάνει την αυστηρότητα της μεθόδου, επιτρέποντας τον εντοπισμό μόνο των ισχυρότερων κύκλων, ενώ μικρότερες τιμές επιτρέπουν περισσότερες αλλά και πιθανώς λανθασμένες ανιχνεύσεις.

- Η επιλογή των παραμέτρων έγινε με πειραματισμό, με βάση την εικόνα εισόδου και το είδος των κύκλων που θέλαμε να ανιχνεύσουμε.

Sobel + Hough

- Η χρήση διαφόρων τιμών κατωφλίου $V_{\min}$ (180, 200, 220, 240, 260) επηρέασε σημαντικά τον αριθμό των ανιχνευόμενων κύκλων.
- Χαμηλότερα $V_{\min}$ έδωσαν υπερβολικά πολλούς κύκλους, συχνά σε άσχετα σημεία, ενώ υψηλότερες τιμές εντόπισαν μόνο τους κύριους κύκλους της μπάλας.
- Προσθέσαμε φίλτρο που αγνοεί κυκλικά κέντρα πολύ κοντινά μεταξύ τους (σε απόσταση < 15 pixels), ώστε να αποφευχθούν επικαλύψεις.

LoG + Hough

- Η έξοδος της LoG παρουσίαζε **μεγαλύτερη πυκνότητα ακμών** σε σχέση με τον Sobel. Αυτό συμβαίνει επειδή ο LoG εντοπίζει **zero crossings**, δηλαδή περιοχές όπου η καμπυλότητα αλλάζει πρόσημο.
- Τέτοιες περιοχές προκύπτουν όχι μόνο στα κυρίως περιγράμματα, αλλά και σε:
 - **μικρές τοπικές μεταβολές φωτεινότητας** (όπως η υφή της μπάλας),
 - **ανωμαλίες φωτισμού** (π.χ. σκιές, αντανάκλασεις),
 - ακόμα και σε **εσωτερικές λεπτομέρειες** που ο Sobel αγνοεί.
- Ο Sobel βασίζεται στην **πρώτη παράγωγο** και ανιχνεύει κυρίως τα περιγράμματα όπου υπάρχει σαφής και έντονη μετάβαση φωτεινότητας. Αντίθετα, ο LoG (μέσω δεύτερης παραγώγου) είναι **πιο ευαίσθητος σε καμπυλότητες** και μπορεί να εντοπίσει πολλά "μικρο-αντικείμενα" ή θόρυβο, αν δεν έχει προηγηθεί φιλτράρισμα.
- Για τον λόγο αυτό, ακόμη και με την εφαρμογή Gaussian blur, η εικόνα ακμών που παράγει ο LoG περιέχει **πολλαπλάσια σημεία**, τα οποία καταλήγουν να δίνουν **περισσότερες ψήφους** στο Hough accumulator — και έτσι **πολλοί περισσότεροι κύκλοι ανιχνεύονται** για ίδια τιμή V_{min} , συγκριτικά με τον Sobel.
- Ως αποτέλεσμα, έπρεπε να χρησιμοποιήσουμε **υψηλότερες τιμές V_{min}** για να μην είναι υπερβολικά πολλοί οι κύκλοι που σχεδιάζονται στην εικόνα. Παρόμοια με τον Sobel, η αύξηση του V_{min} μειώνει τον αριθμό των κύκλων που εντοπίζονται.. Για το LoG, οι τιμές V_{min} που χρησιμοποιήθηκαν ήταν μεγαλύτερες (550, 560, 570, 575, 578) σε σχέση με τον Sobel.

5. ΛΕΙΤΟΥΡΓΙΑ ΣΥΝΑΡΤΗΣΕΩΝ

Συνάρτηση: `fir_conv (in_img_array, mask, in_origin, mask_origin)`

Λειτουργία:

Αυτή η συνάρτηση υλοποιεί την πράξη της συνέλιξης μεταξύ μιας εικόνας και μιας μάσκας, όπως π.χ. το Sobel ή το Laplacian.

Η λειτουργία της βασίζεται στην εξής λογική:

- Για κάθε pixel της εικόνας, η συνάρτηση "τοποθετεί" τη μάσκα πάνω στην εικόνα, με το κέντρο της μάσκας (το `mask_origin`) να συμπίπτει με το συγκεκριμένο pixel.
- Έπειτα, υπολογίζει το άθροισμα των γινομένων ανάμεσα στα στοιχεία της μάσκας και τα αντίστοιχα στοιχεία της εικόνας κάτω από αυτή.
- Το αποτέλεσμα αυτού του αθροίσματος αποθηκεύεται στη νέα εικόνα εξόδου, στην ίδια θέση.
- Επαναλαμβάνει τη διαδικασία για κάθε pixel της εικόνας.
- Το `in_origin` και `mask_origin` καθορίζουν τα σημεία από όπου ξεκινά η σάρωση της εικόνας και της μάσκας αντίστοιχα - δηλαδή ουσιαστικά επιτρέπουν να "κεντράρεις" τη μάσκα στο σωστό σημείο.

Συνάρτηση: `sobel_edge (in_img_array, threshold)`

Λειτουργία:

Αυτή η συνάρτηση εντοπίζει ακμές στην εικόνα εφαρμόζοντας τα Sobel φίλτρα για τις δύο κατευθύνσεις: οριζόντια (x) και κατακόρυφη (y).

Η λειτουργία της βασίζεται στα εξής βήματα:

- Πρώτα, δημιουργεί δύο μάσκες Sobel, μία για τον υπολογισμό της κλίσης ως προς τον x άξονα και μία για τον y.
- Με χρήση της `fir_conv`, εφαρμόζει συνέλιξη της εικόνας με καθεμία από τις δύο μάσκες. Το αποτέλεσμα είναι δύο εικόνες: μία που περιέχει τις οριζόντιες μεταβολές και μία για τις κατακόρυφες.
- Σε κάθε pixel, υπολογίζει το μέτρο του διανύσματος κλίσης, δηλαδή τον συνολικό βαθμό μεταβολής της φωτεινότητας, με βάση τον τύπο:

$$g(n_1, n_2) = \sqrt{g_1(n_1, n_2)^2 + g_2(n_1, n_2)^2}$$

- Στη συνέχεια εφαρμόζει κατώφλι (threshold) στο μέτρο αυτό, ώστε να κρατήσει μόνο τα pixels όπου η μεταβολή είναι αρκετά μεγάλη για να θεωρηθεί ακμή.
- Επιστρέφει την τελική εικόνα ακμών ως δυαδικό πίνακα: 1 εκεί που εντοπίστηκε ακμή, και 0 αλλού.

Με αυτόν τον τρόπο, η `sobel_edge` εντοπίζει τις κύριες ακμές της εικόνας, εστιάζοντας κυρίως στα περιγράμματα και απορρίπτοντας αδύναμες μεταβολές.

Συνάρτηση: `log_edge (in_img_array)`

Λειτουργία:

Η `log_edge` εφαρμόζει τη μέθοδο Laplacian of Gaussian (LoG) για τον εντοπισμό ακμών μέσω zero-crossings — δηλαδή σημείων όπου η φωτεινότητα αλλάζει έντονα και η δεύτερη παράγωγος της εικόνας αλλάζει πρόσημο.

Η λειτουργία της έχει ως εξής:

- Ξεκινά με μια LoG μάσκα 5×5 (προκαθορισμένη), η οποία ενσωματώνει Gaussian εξομάλυνση και Laplacian φίλτρο ταυτόχρονα.
- Χρησιμοποιεί τη `fir_conv` για να εφαρμόσει συνέλιξη της εικόνας με τη LoG μάσκα και να παραχθεί μια νέα εικόνα που εκφράζει τη "καμπυλότητα" κάθε περιοχής.
- Στη συνέχεια, εξετάζει κάθε pixel και την τοπική γειτονιά του, ελέγχοντας αν υπάρχει αλλαγή πρόσημου (zero-crossing) μεταξύ του κεντρικού pixel και των γειτόνων του — αυτό αποτελεί πιθανή ένδειξη ακμής.
- Για να μην ανιχνεύονται τυχαία crossings που οφείλονται σε μικρές διακυμάνσεις (δηλ. θόρυβο), προστίθεται επιπλέον έλεγχος: θεωρείται έγκυρο zero-crossing μόνο αν η διαφορά μεταξύ του μέγιστου και του ελάχιστου της γειτονιάς είναι > 0.5 .
- Τέλος, επιστρέφει μια δυαδική εικόνα ακμών, όπου οι τιμές 1 αντιστοιχούν σε έγκυρα σημεία zero-crossing.

Η `log_edge` είναι πιο ευαίσθητη από το Sobel και μπορεί να ανιχνεύσει λεπτότερες μεταβολές, αλλά είναι πιο επιρρεπής στον θόρυβο — γι' αυτό

συνδυάστηκε με Gaussian blur πριν από την κλήση της συνάρτησης, ώστε να καθαρίσει η είσοδος.

Συνάρτηση: **circ_hough (in_img_array, R_max, dim, V_min)**

Λειτουργία:

Η circ_hough υλοποιεί τη Circular Hough Transform, έναν αλγόριθμο για τον εντοπισμό κύκλων σε εικόνα ακμών.

Η λειτουργία της ακολουθεί τα εξής βήματα:

- Δημιουργεί έναν τρισδιάστατο συσσωρευτή (accumulator) με διαστάσεις που αντιστοιχούν σε πιθανές τιμές για το x-κέντρο, το y-κέντρο και την ακτίνα. Το πλήθος των διακριτών τιμών σε κάθε διάσταση ορίζεται από το dim.
- Υπολογίζει τα εύρη τιμών για τα κέντρα (a, b) και τις ακτίνες (r), χρησιμοποιώντας γραμμική κατανομή (linspace). Οι ακτίνες κυμαίνονται από 20 (R_min) μέχρι R_max.
- Για κάθε pixel της εικόνας που έχει τιμή 1 (δηλαδή έχει χαρακτηριστεί ως ακμή), θεωρεί ότι μπορεί να είναι σημείο ενός κύκλου με ακτίνα r, και για κάθε πιθανή ακτίνα "γυρίζει" τον κύκλο γύρω από αυτό το σημείο (χρησιμοποιώντας το ημιτονοειδές μοντέλο).
- Κάθε τέτοιος κύκλος ρίχνει "ψήφους" (votes) σε όλες τις πιθανές θέσεις του κέντρου του στον συσσωρευτή, αυξάνοντας την τιμή του κελιού (a, b, r) κατά 1.
- Αφού ολοκληρωθούν όλες οι ψήφοι, σαρώνει τον συσσωρευτή και κρατάει μόνο τα σημεία που έχουν πλήθος ψήφων $\geq V_min$. Αυτά θεωρούνται πιθανές ανιχνεύσεις κύκλων.
- Τέλος, επιστρέφει δύο λίστες: τις συντεταγμένες των κέντρων και τις αντίστοιχες ακτίνες των εντοπισμένων κύκλων.

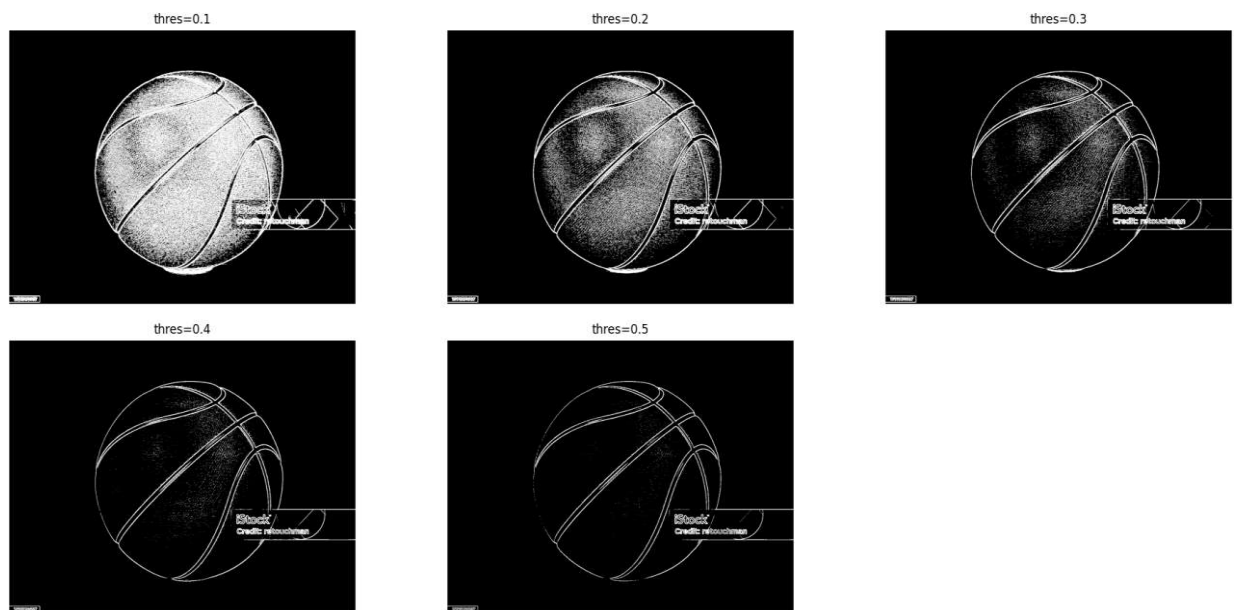
Η circ_hough είναι εξαιρετικά ευαίσθητη στη ποιότητα της εικόνας ακμών, και οι παράμετροι dim, R_max και V_min παίζουν καθοριστικό ρόλο:

- Μεγάλα dim δίνουν μεγαλύτερη ακρίβεια αλλά πιο αργή εκτέλεση,
- Μεγαλύτερο V_min φιλτράρει-απορρίπτει ψευδείς κύκλους,
- Ενώ το R_max καθορίζει ποιο εύρος ακτίνων εξετάζεται.

6. ΠΕΙΡΑΜΑΤΙΚΑ ΑΠΟΤΕΛΕΣΜΑΤΑ

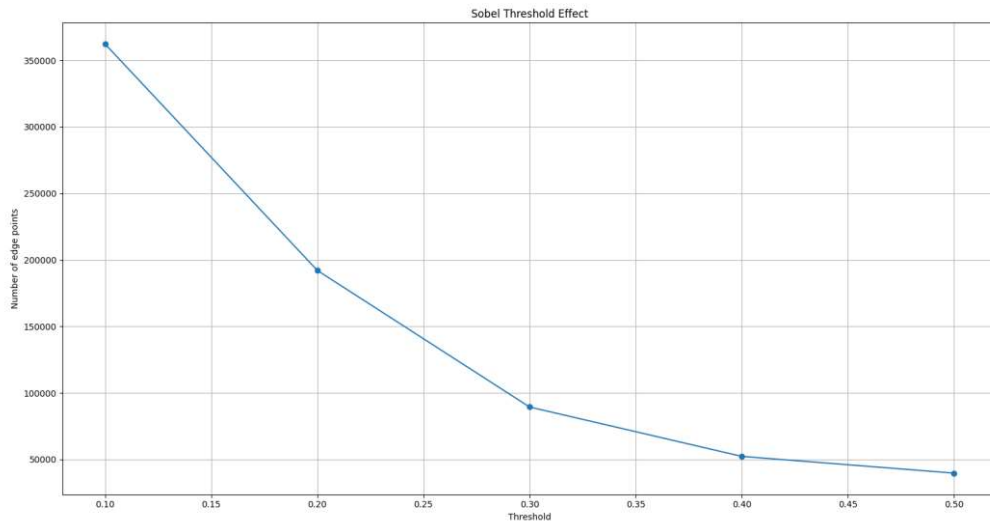
SOBEL ΓΙΑ THRESHOLDS (0.1, 0.2, 0.3, 0.4, 0.5)

Sobel Edge Detection (διάφορα thresholds)



Παρατηρούμε ότι όσο αυξάνεται το Threshold, ο θόρυβος στο εσωτερικό της μπάλας μειώνεται και η εικόνα “καθαρίζει”.

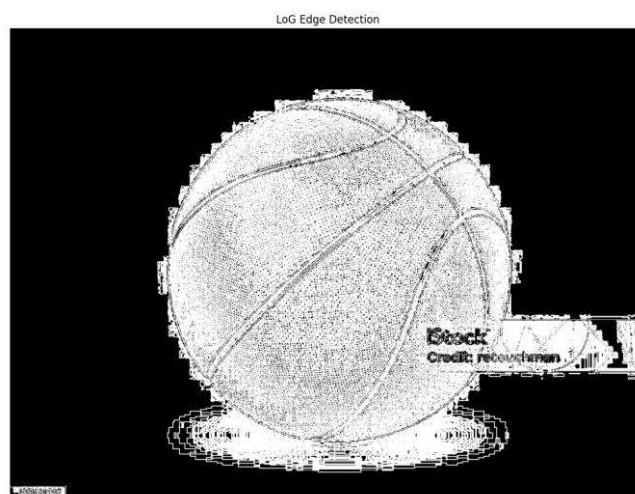
THRESHOLD GRAPH



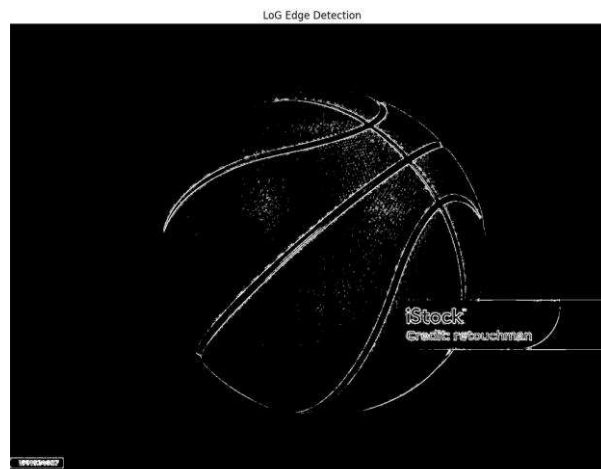
Το διάγραμμα μας δείχνει ότι με την αύξηση του threshold, μειώνεται ο αριθμός των ακμών που εντοπίζονται, καθώς

LoG

ΧΩΡΙΣ GAUSSIAN-BLUR



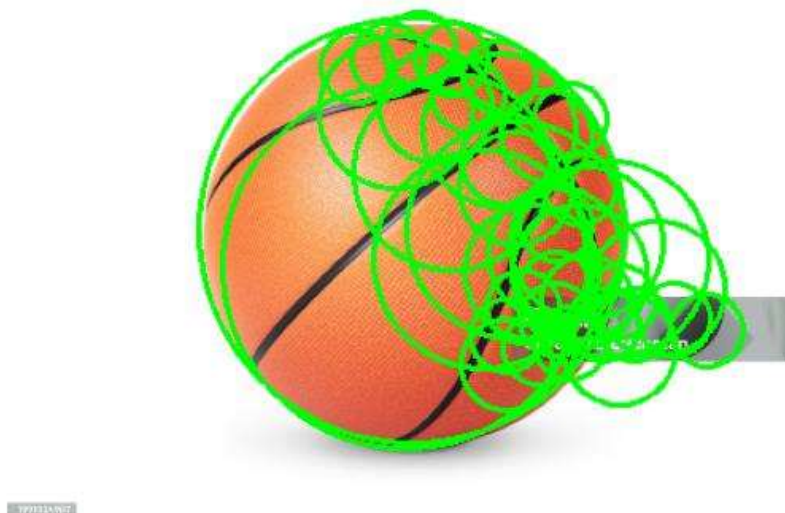
ME GAUSSIAN-BLUR



SOBEL + HOUGH ΓΙΑ V MIN (180, 200, 220, 240, 260)

$$V_{\min} = 160$$

Sobel + Hough ($V_{\min} = 160$)



$V_{\min} = 180$

Sobel + Hough ($V_{\min} = 180$)



$V_{\min} = 200$

Sobel + Hough ($V_{\min} = 200$)



$$V_min = 220$$

Sobel + Hough ($V_min = 220$)



$$V_min = 240$$

Sobel + Hough ($V_min = 240$)



Παρατηρούμε ότι όσο αυξάνεται το V_min μένουν οι πιο αξιόπιστοι κύκλοι, δηλαδή εκείνοι που ψηφίζονται από πολλά pixels και άρα είναι γεωμετρικά πιο πλήρεις και σωστοί.

LoG + HOUGH TIA V_MIN (550, 560, 570, 575, 578)

$V_{\min} = 550$

LoG + Hough ($V_{\min} = 550$)



$$V_{\min} = 560$$

LoG + Hough ($V_{\min} = 560$)



1707135007

$$V_{\min} = 570$$

LoG + Hough ($V_{\min} = 570$)



1707135007

$$V_{\min} = 575$$

LoG + Hough ($V_{\min} = 575$)



$$V_{\min} = 578$$

LoG + Hough ($V_{\min} = 578$)



Ομοίως κι εδώ, με την αύξηση του $V_{\min}$, μένουν οι πιο αξιόπιστοι-”καλοί” κύκλοι.

ΣΗΜΕΙΩΣΕΙΣ :

- Το πρόγραμμα ξεκινάει ζητώντας το path της εικόνας που θέλει να εισάγει ο χρήστης για πειραματισμό, ώστε να δουλεύει για οποιαδήποτε εικόνα κι αν δοθεί.
- Στο terminal εμφανίζονται μηνύματα για κάθε αντίστοιχη ενέργεια που εκτελείται, ώστε να καταλαβαίνει ο χρήστης τι εκτελείται κάθε στιγμή και τι περιμένει να δει. Εμφανίζονται επίσης τα κέντρα και οι ακτίνες για κάθε κύκλο που σχεδιάστηκε στη μέθοδο Hough, όπως επίσης και το πόσοι κύκλοι εντοπίστηκαν σε πλήθος, τους κύκλους δηλαδή που περνάνε το κατώφλι των ελάχιστων ψήφων, για την αντίστοιχη τιμή V_{min} . Ωστόσο δεν σχεδιάζονται όλοι όσοι εντοπίστηκαν λόγω του φίλτρου που βάλαμε στο σχεδιασμό των κύκλων για την αποφυγή πολύ κοντινών-επικαλυπτόμενων κύκλων:

```
if any(np.hypot(x - xc, y - yc) < 15 for xc, yc in drawn):  
    continue  
drawn.append((x, y))
```

- Αν θέλαμε να κρατήσουμε συγκεκριμένους κύκλους, π.χ με ακτίνες απο 30-70, μπορούσαμε να το κάνουμε με ένα φίλτρο ακριβώς πριν σχεδιαστούν οι κύκλοι, όπως για παράδειγμα:
 if not (30 <= r <= 70):
 continue

Ωστόσο δεν θεώρησα ότι χρειάζεται οπότε δεν το έκανα.

- Τα αποτελέσματα στο Sobel + Hough εμφανίζονται σχετικά γρήγορα, περίπου 10 δευτερόλεπτα ανά figure, ωστόσο στο LoG + Hough θέλουν περισσότερη ώρα, περίπου 1 λεπτό ανά figure, παρά το resize της αρχικής εικόνας που έγινε για ταχύτερη εκτέλεση.